

AI TOPIC 13

Artificial Neural Network (ANN) - Complete Theory Guide

তোমার জন্য সব concept একদম বিস্তারিতভাবে বাংলা-ইংরেজি মিশিয়ে বুঝিয়ে দিচ্ছি। Slide এর সব information এখানেই আছে।

১. ANN কী? (What is ANN?)

ANN = Artificial Neural Network

Definition: A collection of networked artificial neurons inspired by the structure of the brain (cerebral cortex), used as a data processing system.

সহজ কথায় — মানুষের মস্তিষ্ক যেভাবে কাজ করে, সেটাকে গণিত দিয়ে কম্পিউটারে বানানোর চেষ্টা হলো ANN।

Artificial Neuron কী?

একটি biological neuron এর mathematical model — অর্থাৎ জীবদেহের নিউরনকে সংখ্যা ও সূত্র দিয়ে প্রকাশ করা।

২. Human Nervous System — কেন জানা দরকার?

ANN বোঝার আগে biological background জানা দরকার।

Human Nervous System দুই ভাগে বিভক্ত:

- **CNS** = Central Nervous System (মস্তিষ্ক + মেরুদণ্ড)
- **PNS** = Peripheral Nervous System (বাকি সব স্নায়ু)

এই system তিন ধরনের কাজ করে:

- **Voluntary** = ইচ্ছাকৃত কাজ (হাত তোলা)
 - **Involuntary** = অনিচ্ছাকৃত (হার্টবিট)
 - **Reflex** = প্রতিবর্ত ক্রিয়া (আঙুলে আগুন লাগলে সরিয়ে নেওয়া)
-

৩. Biological Neuron — এটা আসল নিউরন

[Dendrites] → [Cell Body/SOMA] → [Axon] → [Synapse] → পরের নিউরন

Parts of a Neuron:

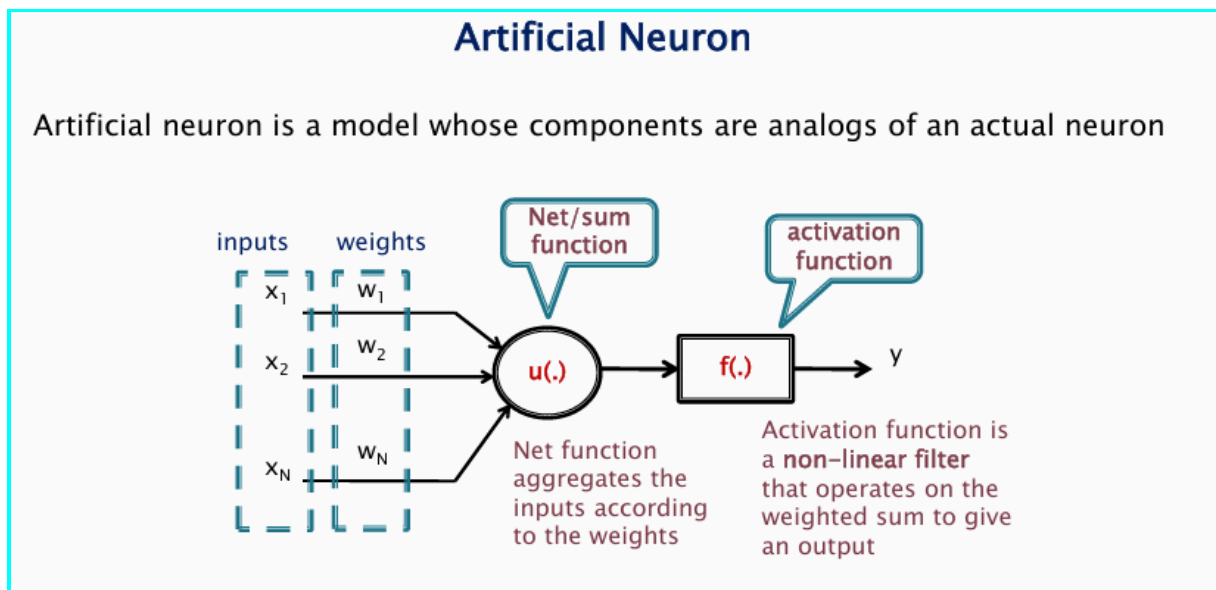
Part	কাজ
Dendrites	অন্য নিউরন থেকে signal গ্রহণ করে
Cell Body (SOMA)	সব signal একত্র করে process করে
Axon	processed signal পরের নিউরনে পাঠায়
Synapse	দুটো নিউরনের মধ্যে সংযোগ বিন্দু

গুরুত্বপূর্ণ নিয়ম:

- নিউরন সব excitatory (উত্তেজক) ও inhibitory (দমনকারী) signal এর sum করে
- যদি excitatory বেশি হয় → নিউরন **"fire"** করে → পরের নিউরনে signal পাঠায়
- এই **fire/no-fire** concept টাই ANN এর ভিত্তি

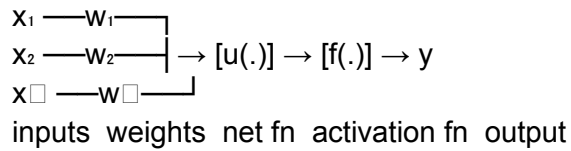
Human Brain এ আছে:

- 10^{11} টি নিউরন
- 10^{14} টি synapse



8. Artificial Neuron — গণিতের ভাষায় নিউরন

এবার biological neuron কে mathematical model এ রূপান্তর করা হয়:



দুটো মূল function:

8.১ Net/Sum Function — $u(.)$

সব weighted input যোগ করে:

$$I = \sum w_i x_i = w_1 x_1 + w_2 x_2 + \dots + w_N x_N$$

Vector আকারে: $I = \mathbf{w}^T \mathbf{x}$

8.২ Activation Function — $f(.)$

Definition: A non-linear filter that operates on the weighted sum to produce an output.

Weighted sum পেলে এই function সিদ্ধান্ত নেয় — output কত হবে।

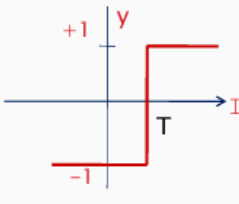
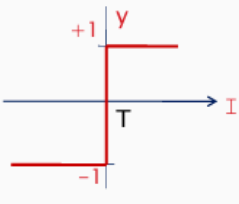
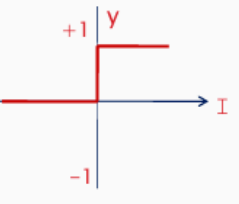
$$\mathbf{y} = \Phi(I - T) \text{ যেখানে } T = \text{threshold}$$

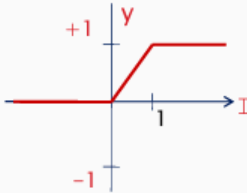
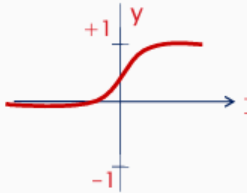
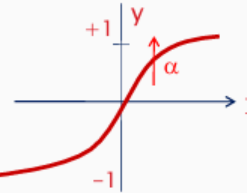
Threshold কে bias দিয়ে replace করলে: $I = \sum w_i x_i + b \quad \mathbf{y} = \Phi(I)$

Artificial Neuron...

Net function and activation function...

Types of activation function...

Threshold	Signum	Step
$y = \Phi(I) = \begin{cases} +1 & I \geq T \\ -1 & I < T \end{cases}$	$y = \Phi(I) = \begin{cases} +1 & I \geq 0 \\ -1 & I < 0 \end{cases}$	$y = \Phi(I) = \begin{cases} +1 & I \geq 0 \\ 0 & \text{otherwise} \end{cases}$
		

Ramp	Unipolar Sigmoid	Bipolar Sigmoid
$y = \Phi(I) = \begin{cases} 1 & I > 1 \\ I & 0 < I \leq 1 \\ 0 & I < 0 \end{cases}$	$y = \Phi(I) = \frac{1}{1 + e^{-\alpha I}}$	$y = \Phi(I) = \frac{2}{1 + e^{-\alpha I}} - 1$
α is slope of the sigmoid		
		

৫. Types of Activation Functions — ৬ ধরনের

Type 1: Threshold (দ্বারপ্রান্ত)

$$y = +1 \text{ যদি } I \geq T$$

$$y = -1 \text{ যদি } I < T$$

Graph এ দেখতে একটা step এর মতো, T বিন্দুতে লাফ দেয়।

Type 2: Signum (চিহ্ন ফাংশন)

$$y = +1 \text{ যদি } I \geq 0$$

$$y = -1 \text{ যদি } I < 0$$

Threshold এরই বিশেষ রূপ যেখানে $T=0$ ।

Type 3: Step (ধাপ ফাংশন)

$$y = +1 \text{ যদি } I \geq 0$$

$y = 0$ অন্যথায়

Type 4: Ramp (ঢাল)

$y = 1$ যদি $|x| > 1$

$y = |x|$ যদি $0 < |x| \leq 1$

$y = 0$ যদি $|x| < 0$

সরল রেখার মতো উঠে তারপর flat হয়।

Type 5: Unipolar Sigmoid (S-আকৃতির, 0 থেকে 1)

$$y = 1 / (1 + e^{(-\alpha|x|)})$$

- Output সবসময় 0 এবং 1 এর মধ্যে
- α = sigmoid এর slope (খাড়াহু)
- সবচেয়ে বেশি ব্যবহৃত activation function

Type 6: Bipolar Sigmoid (-1 থেকে +1)

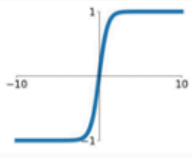
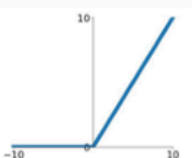
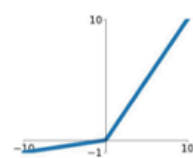
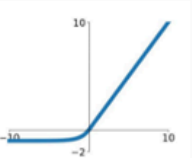
$$y = 2/(1 + e^{(-\alpha|x|)}) - 1$$

- Output সবসময় -1 এবং +1 এর মধ্যে

Artificial Neuron...

Net function and activation function...

Some more types of activation function... Exponential Linear Unit

tanh	ReLU	Leaky ReLU	ELU
$\tanh(x)$	$\max(0, x)$	$\max(0.1x, x)$	$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$
			

Rectified Linear Unit

Maxout
 $\max(w_1^T x + b_1, w_2^T x + b_2)$

hter900@gmail.com 15

৬. আরও কিছু Modern Activation Functions

Function	Formula	বৈশিষ্ট্য
tanh	$\tanh(x)$	-1 থেকে +1, bipolar sigmoid এর মতো
ReLU	$\max(0, x)$	সহজ, deep learning এ জনপ্রিয়
Leaky ReLU	$\max(0.1x, x)$	ReLU এর উন্নত রূপ
ELU	x যদি $x \geq 0$; $\alpha(e^x - 1)$ যদি $x < 0$	Exponential Linear Unit
Maxout	$\max(w_1^T x + b_1, w_2^T x + b_2)$	দুটো linear এর maximum

৭. Worked Example — Artificial Neuron

দেওয়া আছে:

- Activation function: $y = +1$ যদি $I \geq 0$, $y = -1$ যদি $I < 0$
- $w = [1, -1, 0.5]^T$
- $x = [1, -2, 0.5]^T$

সমাধান:

$$I = w^T x = (1 \times 1) + (-1 \times -2) + (0.5 \times 0.5) = 1 + 2 + 0.25 = 3.25$$

যেহেতু $I = 3.25 \geq 0$, তাই $y = +1$ ✓

Artificial Neural Network (ANN)

ANN is a **data processing system** comprising a **collection of artificial neurons networked in a structure** that is inspired by and modeled after the human brain.

ANN is characterized by

1. The model of the neuron
2. The model of the synapses interconnection
3. Learning/Training rule

The model of the neuron is characterized by the **net/sum function** and the **activation function**.

৮. ANN এর তিনটি মূল বৈশিষ্ট্য

ANN কে তিনটি জিনিস দিয়ে সম্পূর্ণভাবে describe করা যায়:

1. **Model of the Neuron** → Net function + Activation function
2. **Model of Synaptic Interconnection** → কীভাবে নিউরনগুলো সংযুক্ত (Architecture)
3. **Learning/Training Rule** → Weight কীভাবে adjust হয়

৯. ANN Connections — দুই ধরনের Architecture

৯.১ Feedforward Network

Input Layer → Hidden Layer → Output Layer

(→ একদিকেই যায়, পিছনে আসে না)

- তথ্য শুধু সামনের দিকে যায়
- Example: $4 \times 3 \times 1$ network মানে → 4টি input, 3টি hidden neuron, 1টি output

৯.২ Recurrent Network

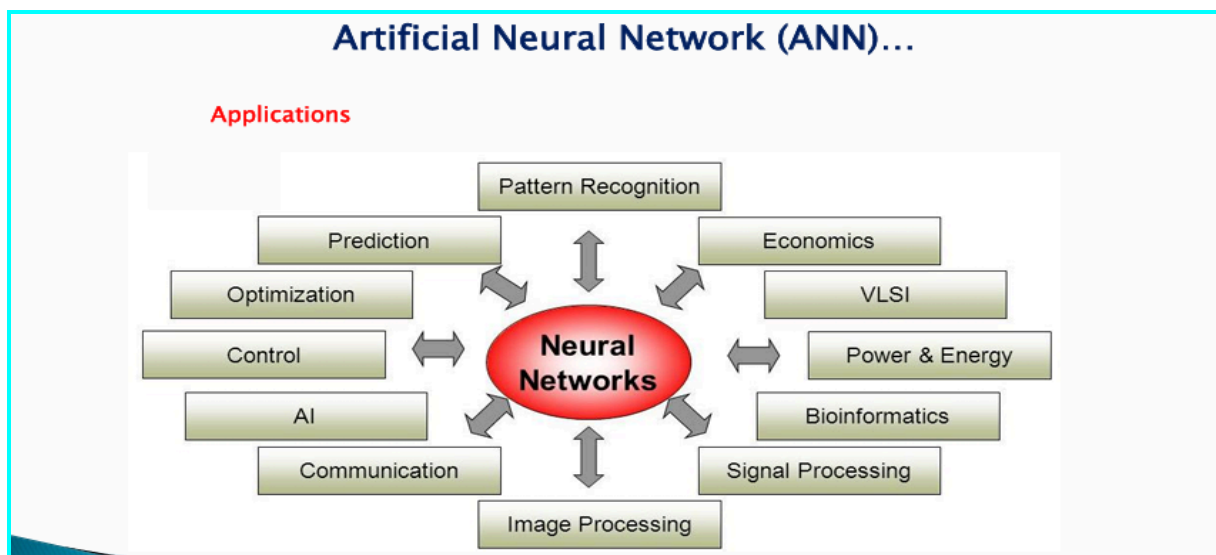
Input → [Neurons] → Output

↑_____↓ (feedback আছে!)

- নিউরনের output আবার নিজেই বা আগের নিউরনে ফেরত যায়
- Time-series, speech এর মতো sequential data তে কার্যকর

১০. ৬ ধরনের ANN (Machine Learning এ ব্যবহৃত)

1. **Feedforward Neural Network** — সবচেয়ে সহজ, একদিকে flow
2. **Radial Basis Function (RBF) Network** — distance-based activation
3. **Kohonen Self Organizing Network** — unsupervised, clustering এ
4. **Recurrent Neural Network (RNN) / LSTM** — sequence data, language
5. **Convolutional Neural Network (CNN)** — image recognition
6. **Modular Neural Network** — ছোট ছোট network মিলে তৈরি



১১. Learning কী? (Training Rule)

ANN দুটো কাজ করে:

১. Learning (শেখা)

Definition: এটি একটি ধারাবাহিক প্রক্রিয়া (**Iterative process**)। যখন আমরা নেটওয়ার্ককে কোনো ইনপুট দেই, সে শুরুতে ভুল রেজাল্ট দিতে পারে। তখন সিস্টেমটি ধীরে ধীরে তার **Weights** গুলো পরিবর্তন করতে থাকে যতক্ষণ না সে সঠিক আউটপুট দিতে পারছে।

- **Biological Analogy:** আমাদের মস্তিষ্কে দুটি নিউরনের সংযোগস্থলকে বলা হয় **Synapse**। আমরা যখন নতুন কিছু শিখি, তখন এই সিন্যাপসের শক্তি (Synaptic strength) বাড়ে বা কমে। ANN-এ এই সিন্যাপসের শক্তিই হলো **Weight**।
- কাজ: Weight পরিবর্তন করার অর্থ হলো কোন ইনপুটটিকে বেশি গুরুত্ব দিতে হবে আর কোনটিকে কম, তা ঠিক করা।

২. Recall (মনে করা/ব্যবহার করা)

যখন মডেলটি শেখা শেষ করে (Training complete), তখন তাকে নতুন কোনো ডাটা দিলে সে তার জমানো অভিজ্ঞতা (Trained Weights) ব্যবহার করে সাথে সাথে রেজাল্ট দিয়ে দেয়। একেই বলে **Recall**।

- উদাহরণ: পরীক্ষার আগে পড়া হলো **Learning**, আর পরীক্ষার হলে উত্তরপত্র লেখা হলো **Recall**।

৩. দুই ধরনের Training (বিস্তারিত)

বৈশিষ্ট্য	Supervised (সুপারভাইজড)	Unsupervised (আন-সুপারভাইজড)
শিক্ষকের ভূমিকা	এখানে একজন "শিক্ষক" বা Labeled Data থাকে। অর্থাৎ ইনপুট দেওয়ার সাথে সাথে বলে দেওয়া হয় সঠিক আউটপুট কী হবে।	এখানে কোনো শিক্ষক বা লেবেল থাকে না। নেটওয়ার্ককে শুধু ডাটা দেওয়া হয়।
প্রক্রিয়া	মডেল একটি আউটপুট দেয়, তারপর সেটিকে আসল উত্তরের সাথে তুলনা করা হয়। যদি ভুল হয়, তবে Error বের করে Weight কমানো বা বাড়ানো হয়।	নেটওয়ার্ক নিজেই ডাটার মধ্যে মিল বা Patterns/Clusters খুঁজে বের করার চেষ্টা করে।
উদাহরণ	Image Classification: বিড়ালের ছবি দিয়ে বলে দেওয়া হলো "এটি বিড়াল"। পরে সে বিড়াল চিনতে শেখে।	Clustering: ১০০০ জন ফ্রেতার ডাটা দিলে মডেল নিজে থেকেই তাদের খরচের ধরন অনুযায়ী ৩টি গ্রুপে ভাগ করে ফেলল।

সংক্ষেপে মনে রাখার জন্য:

- **Supervised:** "ইনপুট হলো $\$X\$$, আউটপুট হতে হবে $\$Y\$$ । ভুল করলে বকা (Error) থাকে এবং নিজেকে শুধরে নেবে।"
- **Unsupervised:** "এই নাও অনেকগুলো ডাটা, এদের মধ্যে কী মিল আছে তুমি নিজেই খুঁজে বের করো।"

১২. Perceptron — সবচেয়ে সহজ ANN

Perceptron হলো নিউরাল নেটওয়ার্কের একদম প্রাথমিক ধাপ। এটি মূলত একটি গাণিতিক ফাংশন যা ইনপুট নিয়ে একটি বাইনারি সিদ্ধান্ত দেয়। তোমার দেওয়া পয়েন্টগুলো নিচে আরও ক্লিয়ার করে দিচ্ছি:

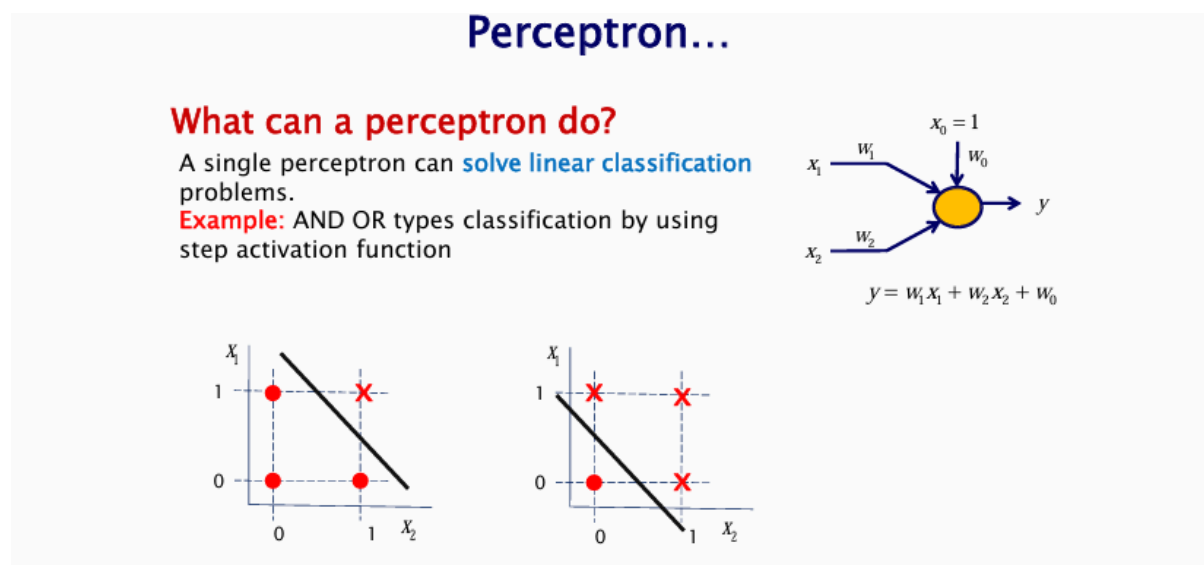
১২. Perceptron — সবচেয়ে সহজ ANN

Definition: এটি মস্তিষ্কের সেই ক্ষমতাকে নকল করে যার মাধ্যমে আমরা কোনো কিছু দেখে চিনতে পারি বা আলাদা করতে পারি।

- সহজ কথায়: এটি হলো একটি **Single Layer Neural Network** যেখানে মাত্র একটি নিউরন থাকে। এটি মূলত একটি **Binary Classifier** হিসেবে কাজ করে।

১৩. Perceptron কী কী সমাধান করতে পারে?

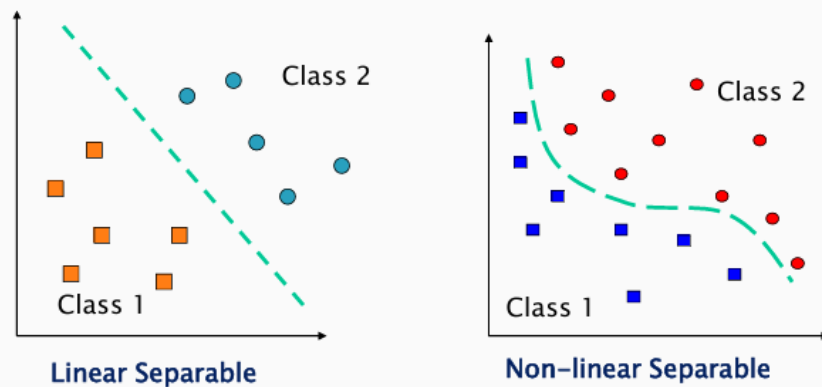
Linear Classification ✓



Non-linear Classification — XOR সমস্যা ✗

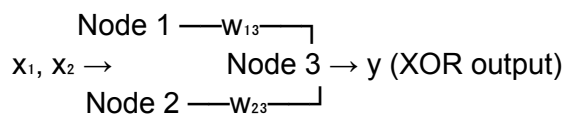
XOR gate একটা straight line দিয়ে আলাদা করা যায় না। এজন্য **multi-layer perceptron** দরকার।

Linearly Separable and Non-linearly Separable Problem



(কোনো একটা line দিয়ে X এবং • আলাদা করা সম্ভব না)

XOR সমাধান: দুটো hidden node ব্যবহার করে (2-layer network):

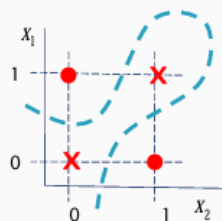
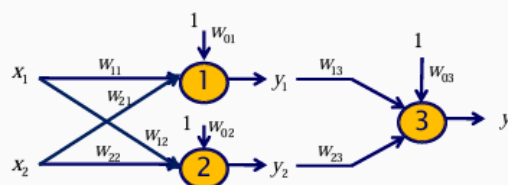


XOR এর জন্য **weight solution**:

- $W_{11} = W_{13} = W_{23} = W_{22} = +1$
- $W_{21} = -1; W_{12} = -1$
- $W_{01} = -1; W_{02} = -1; W_{03} = -1$

Perceptron...

Let's try two nodes for the EXCLUSIVE-OR classification



১৪. Perceptron Learning Rule — কীভাবে weight শেখে?

Weight Adjustment এর General Formula:

$$w(k) = w(k-1) + \Delta w(k)$$

যেখানে weight adjustment: $\Delta w(k) = \eta \cdot r(k) \cdot x(k)$

- η (eta) = Learning constant (কত দ্রুত শিখবে, সাধারণত 0 থেকে 1)
- r = Learning signal = $d - y$ (desired output - actual output)
- x = input vector

Perceptron Learning Rule বিস্তারিত:

$$\Delta w_i = \eta [d_i - y_i] x_i$$

Signum activation এ:

$$\Delta w_i = 2\eta \cdot d_i \cdot x_i \quad \text{যদি } y_i \neq d_i \text{ (ভুল হলে correct করো)}$$

$$= 0 \quad \text{otherwise (ঠিক থাকলে কিছু করো না)}$$

১৫. Perceptron Learning — Worked Example

দেওয়া আছে:

- $\eta = 0.1$
- Initial weights: $w(1) = [1, -1, 0, 0.5]^T$
- Training data:
 - $x(1) = [1, -2, 0, 1]^T$, $d(1) = -1$
 - $x(2) = [0, 1.5, -0.5, 1]^T$, $d(2) = -1$
 - $x(3) = [-1, 1, 0.5, 1]^T$, $d(3) = +1$

Step 1: $x(1)$ দিয়ে শুরু

$$l(1) = w^T x = [1, -1, 0, 0.5] \cdot [1, -2, 0, 1]^T = 1 + 2 + 0 + 0.5 = 3.5$$

$$y(1) = \text{sgn}(3.5) = +1$$

কিন্তু $d(1) = -1$, তাই ভুল হয়েছে!

$$r(1) = d(1) - y(1) = -1 - 1 = -2$$

$$\Delta w = \eta \cdot r \cdot x = 0.1 \times (-2) \times [1, -2, 0, 1]^T = [-0.2, 0.4, 0, -0.2]^T$$

$$w(2) = w(1) + \Delta w = [1, -1, 0, 0.5]^T + [-0.2, 0.4, 0, -0.2]^T = [0.8, -0.6, 0, 0.3]^T$$

Step 2: $x(2)$ দিয়ে

$$l(2) = [0.8, -0.6, 0, 0.3] \cdot [0, 1.5, -0.5, 1]^T = 0 - 0.9 + 0 + 0.3 = -0.6$$

$$y(2) = \text{sgn}(-0.6) = -1$$

$$d(2) = -1, \text{ তাই ঠিক আছে! } r(2) = -1 - (-1) = 0$$

$$\Delta w(2) = 0, \text{ তাই } w(3) = w(2) = [0.8, -0.6, 0, 0.3]^T$$

Step 3: $x(3)$ দিয়ে — নিজে করো

$$r(3) = 1 - (-1) = 2 \rightarrow \Delta w(3) = 0.1 \times 2 \times [-1, 1, 0.5, 1]^T$$

$$\text{Final answer: } w(4) = [0.6, -0.4, 0.1, 0.5]^T$$

১৬. ADALINE Learning Rule

ADALINE = Adaptive Linear Element

Perceptron vs ADALINE পার্থক্য:

বিষয়	Perceptron	ADALINE
Activation	Hard-limiting (± 1 step)	Linear (সরাসরি I এর মান)
Error calculation	quantized output দিয়ে	continuous output দিয়ে

Goal:

$$\text{Weight খুঁজে বের করা যাতে: } y = \sum w_i x_i = d$$

Cost Function (Error Function):

$$E(w) = \frac{1}{2} \sum (d(k) - y(k))^2 = \frac{1}{2} \sum (d(k) - w^T x(k))^2$$

Gradient Descent Algorithm:

Error কমাতে weight কে negative gradient এর দিকে সরাই:

$$\Delta w = -\eta \nabla E(w)$$

এর ফলে:

$$\Delta w = \eta(d(k) - w^T(k)x(k))x(k)$$

এটাই — **ADALINE / Widrow-Hoff / LMS (Least Mean Square) Rule**

ADALINE Step 1 Example (আগের data দিয়ে):

$l(1) = 3.5$, $y(1) = 3.5$ (linear, কোনো activation নেই)

$$r = d(1) - y(1) = -1 - 3.5 = -4.5...$$

আসলে ADALINE এ: $\Delta w = 0.1 \times (-1 - 3.5) \times [1, -2, 0, 1]^T = 0.1 \times (-4.5) = [-0.45, 0.9, 0, -0.45]^T$

তাই ADALINE step size বড় হয় — দ্রুত শেখে কিন্তু oscillate করতে পারে।

১৭. Delta Learning Rule

Perceptron → hard-limiting activation ADALINE → linear activation **Delta Rule** → continuous & differentiable activation (যেমন sigmoid)

Cost Function:

$$E(w) = \frac{1}{2} \sum_k \sum_i [d(k)_i - \Phi(l(k)_i)]^2$$

Weight Update:

$$\Delta w_i = \eta[d(k)_i - \Phi(l(k)_i)] \cdot \Phi'(l(k)_i) \cdot x(k)_i$$

এখানে $\Phi'(l)$ হলো activation function এর derivative।

Sigmoid Derivatives:

Unipolar sigmoid এর জন্য: $\Phi(l) = 1/(1+e^{-l})$ হলে, $\Phi'(l) = y(1-y)$

(এটা prove করা যায় chain rule দিয়ে — $y \times (1-y)$ অনেক সহজ form)

Bipolar sigmoid এর জন্য: $\Phi(l) = 2/(1+e^{-l}) - 1$ হলে, $\Phi'(l) = \frac{1}{2}(1-y^2)$

১৮. তিনটি Learning Rule এর Summary

Rule	Weight Update	Activation
------	---------------	------------

Perceptron	$\Delta w = \eta(d-y)x$	Hard-limiting (step)
ADALINE	$\Delta w = \eta(d-y)x$	Linear
Delta	$\Delta w = \eta(d-y)\Phi'(l)x$	Smooth (sigmoid)

সবার ক্ষেত্রে সাধারণ form: $\mathbf{w}(k+1) = \mathbf{w}(k) + \Delta \mathbf{w}(k)$

পার্থক্য শুধু $\Delta w(k)$ calculate করার পদ্ধতিতে।

১৯. Analytical vs Learning Rule — কোনটা কখন?

পদ্ধতি

কখন ব্যবহার

Analytical Method Input কম থাকলে; manually weight বসানো যায়

Learning Rule Input অনেক বেশি হলে; automatically শেখে

XOR example এ **analytically weight** বের করা হয়েছে। বাস্তব জীবনে (যেমন image recognition এ 784 input) analytical method সম্ভব না — তখন learning rule লাগবেই।

পারসেপট্রন বা নিউরাল নেটওয়ার্ক কীভাবে শেখে, তার দুটি প্রধান পথ আছে। তোমার নোট অনুযায়ী **Analytical method** এবং **Learning Rule**-এর বিস্তারিত নিচে দেওয়া হলো:

১. Analytical Method (গাণিতিক বা বিশ্লেষণমূলক পদ্ধতি)

এটি মূলত সরাসরি অংক কষে ওজনের (Weights) মান বের করার একটি পদ্ধতি।

- কীভাবে কাজ করে: এখানে আমরা আউটপুট ইকুয়েশনগুলোকে এক সেট লিনিয়ার ইকুয়েশন হিসেবে ধরি এবং সেগুলো সমাধান করে w (weights)-এর মান বের করি।
- সীমাবদ্ধতা (**Small inputs only**): * যদি ইনপুট মাত্র ২-৩টি হয়, তবে এটি সহজে করা যায়।
 - কিন্তু যদি ইনপুট সংখ্যা বেড়ে যায় (যেমন ১০০ বা ১০০০ টি), তখন এই ইকুয়েশনগুলো সমাধান করা অসম্ভব হয়ে পড়ে।
 - এটি "একবারই" (One-shot) সমাধান করার চেষ্টা করে, তাই ডাটা বড় হলে এটি আর কার্যকর (Practical) থাকে না।

২. Learning Rule (শিক্ষণ নিয়ম বা অ্যালগরিদম)

এটি হলো একটি **Iterative** বা বারবার চেষ্টার মাধ্যমে শেখার পদ্ধতি। বর্তমানের সব আধুনিক AI (যেমন GPT, BERT) এই পদ্ধতিতে শেখে।

- কেন এটি গুরুত্বপূর্ণ? যখন ইনপুট অনেক বেশি থাকে বা নেটওয়ার্ক অনেক বড় হয়, তখন আমরা সরাসরি অংক না কষে "ভুল থেকে শেখার" পদ্ধতি ব্যবহার করি।

Perceptron Learning Rule কীভাবে কাজ করে? (ধাপসমূহ):

১. **Initialization:** শুরুতে সব Weights (\$w\$) র‍্যান্ডমলি (যেমন: ০ বা ০.৫) ধরে নেওয়া হয়।

২. **Input:** নেটওয়ার্ককে একটি ইনপুট দেওয়া হয়।

৩. **Calculation:** নেটওয়ার্ক তার বর্তমান ওয়েট দিয়ে একটি আউটপুট (\$y\$) বের করে।

৪. **Comparison:** এই আউটপুটকে আসল উত্তরের (Target) সাথে তুলনা করা হয়।

৫. **Adjustment:** যদি উত্তর ভুল হয়, তবে নিচের সূত্র অনুযায়ী ওয়েট আপডেট করা হয়:

$$w_{\text{new}} = w_{\text{old}} + \Delta w$$

$$\text{যেখানে } \Delta w = \eta (\text{target} - \text{output}) \times \text{input}$$

(η হলো **Learning Rate**, যা ঠিক করে কত দ্রুত মডেলটি শিখবে)।

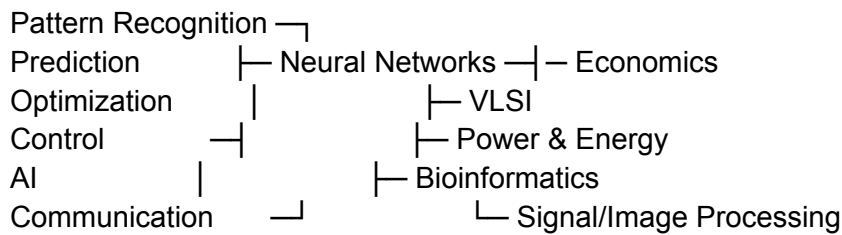
Analytical vs Learning Rule: প্রধান পার্থক্য

বৈশিষ্ট্য	Analytical Method	Learning Rule (Algorithm)
কাজের ধরন	একবারে সরাসরি সমাধান (Exact solution)।	বারবার চেষ্টার মাধ্যমে উন্নতি (Iterative)।
ডাটা সাইজ	শুধুমাত্র ছোট ডাটার জন্য প্রযোজ্য।	অনেক বড় এবং জটিল ডাটার জন্য সেরা।
জটিলতা	ডাটা বাড়লে ক্যালকুলেশন অসম্ভব হয়ে যায়।	এটি বড় নেটওয়ার্ক বা ডিপ লার্নিং-এর ভিত্তি।
নমনীয়তা	নতুন ডাটা আসলে আবার শুরু থেকে অংক করতে হয়।	নতুন ডাটা দিয়ে সহজেই নিজেকে আপডেট করতে পারে।

সহজ উদাহরণ:

- **Analytical Method:** যেন একটা অংকের উত্তর সরাসরি ক্যালকুলেটরে বের করা। ইনপুট ছোট হলে সহজ, কিন্তু ১০০০ ভ্যারিয়েবল থাকলে ক্যালকুলেটর হ্যাং হয়ে যাবে।
- **Learning Rule:** যেন একটা বাম্বা সাইকেল চালানো শিখছে। সে বারবার পড়ে যাচ্ছে (Error), আর প্রতিবার সে বুঝতে পারছে তার ব্যালেন্স (Weight) কোন দিকে ঠিক করতে হবে। এভাবে হাজার বার চেষ্টার পর সে শিখে যাচ্ছে।

২০. ANN Applications



২১. Advantages & Disadvantages

✓ সুবিধা:

- Linear program যা করতে পারে না তাও করতে পারে
- Parallel nature — একটা অংশ নষ্ট হলেও কাজ চলে
- Reprogramming লাগে না — নিজেই শেখে
- Complex problem solve করতে পারে

✗ অসুবিধা:

- Training ছাড়া কাজ করে না
- বড় network এ processing time বেশি লাগে
- Traditional processor architecture এর সাথে মেলে না — emulation লাগে

২২. Real-life Example: Fruit Sorting Machine

Input: $x = [\text{shape}, \text{texture}, \text{weight}]^T$

Encoding:

shape = +1 (round), -1 (elliptical)

texture = +1 (smooth), -1 (rough)

weight = +1 (>1kg), -1 (<1kg)

Prototype:

banana = $[-1, +1, ?]^T \rightarrow$ elliptical, smooth

apple = $[+1, -1, ?]^T \rightarrow$ round, rough

এই input দিয়ে perceptron train করলে সে শিখে যাবে কোনটা apple আর কোনটা banana।

Artificial Neural Network (ANN) মানুষের মস্তিষ্কের নিউরনের আদলে তৈরি একটি গাণিতিক সিস্টেম। এর সুবিধা ও অসুবিধাগুলো সহজভাবে নিচে বুলিয়ে দিচ্ছি:

Advantages (সুবিধা)

- **Non-linear Performance:** প্রথাগত লিনিয়ার (সরলরৈখিক) প্রোগ্রাম দিয়ে সব কাজ হয় না। যেমন মানুষের হাতের লেখা চেনা বা ফেস ডিটেকশন—এসব জটিল কাজ ANN খুব ভালো পারে।
- **Fault Tolerance (প্যারালেল নেচার):** নিউরাল নেটওয়ার্কের একটি বড় অংশ একসাথে (Parallel) কাজ করে। তাই যদি নেটওয়ার্কের একটি ছোট অংশ বা এলিমেন্ট কাজ করা বন্ধ করে দেয়, পুরো সিস্টেমটি ধসে পড়ে না; এটি কাজ চালিয়ে যেতে পারে।
- **Self-learning:** একে বারবার নতুন করে কোড লিখে ইনস্ট্রাকশন দিতে হয় না। এটি ডাটা থেকে নিজে নিজেই প্যাটার্ন শিখে নেয় (Learning by example)।
- **Easy Implementation:** একবার মডেল আর্কিটেকচার ঠিক হয়ে গেলে, বিভিন্ন লাইব্রেরি (যেমন TensorFlow বা PyTorch) ব্যবহার করে এটি বাস্তবায়ন করা সহজ।
- **Robust & Adaptive:** এটি খুব নমনীয়। ডাটা বা পরিবেশ বদলালে এটি নিজেকে মানিয়ে নিতে পারে এবং অত্যন্ত জটিল সমস্যা সমাধানে দক্ষ।
- **Efficiency:** বিজ্ঞানীরা মনে করেন, এর কিছু ঝুঁকি থাকলেও বড় প্রজেক্টের ক্ষেত্রে এর কার্যকারিতা এবং উপযোগিতা অনেক বেশি।

Disadvantages (অসুবিধা)

- **Requires Training:** এটি জাদুর মতো কাজ করে না। একে কাজ শেখানোর জন্য প্রচুর পরিমাণ ডাটা দিয়ে অনেকক্ষণ ধরে 'Training' করাতে হয়।
- **High Processing Time:** বড় বড় নিউরাল নেটওয়ার্ক (যেমন Deep Learning) ট্রেন করতে অনেক সময় এবং শক্তিশালী প্রসেসর (GPU/TPU) লাগে। এটি সাধারণ কম্পিউটারে অনেক সময়সাপেক্ষ।
- **Hardware Dependency (Emulation):** আমাদের বর্তমান কম্পিউটার প্রসেসরগুলো (CPU) যেভাবে তৈরি, নিউরাল নেটওয়ার্কের কাজ করার ধরন তার থেকে আলাদা। তাই হার্ডওয়্যারের পূর্ণ ক্ষমতা ব্যবহার করতে এদের 'Emulate' বা বিশেষ ভাবে চালাতে হয়।
- **Black Box Nature (একটি বাড়তি পয়েন্ট):** নিউরাল নেটওয়ার্ক কীভাবে একটি সিদ্ধান্ত নিল, তার ভেতরের গাণিতিক যুক্তি অনেক সময় মানুষের পক্ষে বোঝা কঠিন হয়ে পড়ে। একে "Black Box" সমস্যা বলা হয়।

সহজ ইনটুইশন:

ANN হলো একটি বাচ্চার মতো।

- সুবিধা: সে নিজে নিজে শেখে, একটা ভুল করলে পুরো পড়া ভুলে যায় না এবং বড় হয়ে খুব জটিল অংক করতে পারে।
- অসুবিধা: তাকে শেখাতে অনেক সময় দিতে হয় এবং তার জন্য দামী গৃহশিক্ষক (High Processing) প্রয়োজন হয়।

English Definitions (Simple & Clear)

- **Artificial Neural Network (ANN):** An ANN is a computer system inspired by the human brain. It consists of layers of interconnected "neurons" that work together to process data and recognize patterns.
- **Activation Function:** It is a mathematical formula that decides whether a neuron should "fire" (be activated) or not based on the input it receives. It acts as a gatekeeper.
- **Threshold:** A threshold is a specific value or "limit." If the input to a neuron reaches or exceeds this value, the neuron sends information to the next layer. If it's below the limit, it stays silent.

১. Artificial Neural Network (ANN) - এটি মূলত কী?

এটি হলো এমন একটি সফটওয়্যার স্ট্রাকচার যা মানুষের মাথার নিউরনের মতো কাজ করার চেষ্টা করে।

- কাজ: এর প্রধান কাজ হলো ডাটার ভেতর থেকে জটিল প্যাটার্ন খুঁজে বের করা। যেমন: একটা ছবি দেখে চিনে ফেলা এটা বিড়াল নাকি কুকুর, অথবা কারোর গলার স্বর শুনে মানুষটিকে চেনা। এটি মূলত ইনপুট নেয়, সেটিকে প্রসেস করে এবং একটি প্রেডিকশন বা রেজাল্ট দেয়।

২. Activation Function - এর কাজ কী?

অ্যাক্টিভেশন ফাংশন হলো নিউরনের ভেতরে থাকা একটি "সিদ্ধান্ত গ্রহণকারী" অংশ।

- কাজ: নিউরনের কাছে যখন কোনো তথ্য আসে, অ্যাক্টিভেশন ফাংশন হিসাব করে দেখে সেই তথ্যটি কি গুরুত্বপূর্ণ? যদি গুরুত্বপূর্ণ হয়, তবেই সে নিউরনটিকে "অ্যাক্টিভেট" করে দেয় যাতে তথ্যটি পরবর্তী লেয়ারে যেতে পারে। এটি মূলত সিস্টেমকে জটিল ক্যালকুলেশন করতে এবং সঠিক আউটপুট দিতে সাহায্য করে।

৩. Threshold (থ্রেশহোল্ড) - এটি কী?

থ্রেশহোল্ডকে তুমি একটি "পাসিং মার্ক" বা সীমানা হিসেবে চিন্তা করতে পারো।

- কাজ: ধরো একটি নিউরনকে বলা হলো, যদি ইনপুট ৫-এর বেশি হয় তবেই তুমি সিগন্যাল পাঠাবে। এখানে '৫' হলো থ্রেশহোল্ড। ইনপুট ৪ হলে নিউরনটি চুপ থাকবে, কিন্তু ৬ হলেই সে সক্রিয় হয়ে উঠবে। এটি নিশ্চিত করে যে সামান্য নয়েজ বা ফালতু তথ্য যেন পুরো সিস্টেমকে ডিস্টার্ব না করে।